

信息科学与工程学院
课程设计报告

课程名称: 网络工程课程设计

题　　目: 基于 NB-IoT 和 LSTM 的智能环境监测预测系统

专业班级: 22 级计算机科学与技术 2 班

学生姓名: 顾凯琪、徐豪城

学　　号: 226001624 、226001662

指导教师: 江先亮

开始时间: 2024 年 9 月 11 日

结束时间: 2024 年 12 月 25 日

摘要

本次课程设计的主题为基于 NB-IoT 的无线温湿度监控预测系统设计与实现，涵盖了温湿度监测在农业、物流、仓储、医疗等多个重要领域的应用。项目团队聚焦于无线监测节点的设计，结合 ESP8266、SIM7020 模块和 DHT22 模块，实现了 NB-IoT 联网与温湿度数据的高精度获取。此外，系统的前后端协同工作也在项目中占据了重要地位，成功实现了从硬件到前后端的全面开发，包括数据存储与渲染，并完成了一系列功能需求，展现了系统功能的复杂性与可用性。

在硬件设计方面，ESP8266 提供了智能控制和数据处理功能，SIM7020 模块确保了监测节点的 NB-IoT 联网能力，实现了高效且可靠的远程通信。结合 DHT22 模块，实现了环境温湿度信息的精准采集，为系统提供了全面的环境感知能力。这一节点设计不仅是系统的核心部分，也为整个监测系统的稳定运行奠定了坚实基础。

在软件平台方面，团队采用了 Visual Studio Code、Arduino IDE 和 IDEA 等工具，分别用于前端代码的编写与测试、硬件代码的编写与烧录，以及后端代码的开发。这些工具的高效应用使得各模块的开发进展顺利，体现了团队在物联网开发中的专业技能。同时，团队成功配置了云服务器，安装了数据库，并搭建了 MQTT 服务器用于消息汇总与传递，进一步深化了对云计算和物联网结合的理解，为系统的可扩展性和数据处理提供了有力支持。

此外，团队还引入了 LSTM 模型对温湿度数据进行预测。通过对历史数据的训练和预测，LSTM 模型能够有效预估未来的温湿度变化，从而为系统提供了更具前瞻性的决策支持。

综上所述，本项目的成功实施不仅是对所学知识的应用，更是对团队整体能力的提升。从硬件设计到前后端开发，再到温湿度预测的实现，团队积累了丰富的物联网开发经验，并培养了解决实际问题的能力。

关键词：NB-IoT　ESP8266 模块MQTT 消息队列　前后端分离　LSTM 模型

Abstract

The theme of this project is the design and implementation of a wireless temperature and humidity monitoring and prediction system based on NB-IoT, with applications in various important fields such as agriculture, logistics, warehousing, and healthcare. Our team focused on the design of wireless monitoring nodes, integrating the ESP8266, SIM7020 module, and DHT22 sensor, successfully achieving NB-IoT connectivity and high-precision temperature and humidity data collection. Furthermore, the collaboration between the front-end and back-end was essential in this project. We developed a complete system from hardware to front-end and back-end, including data storage and rendering, and successfully implemented a series of functional requirements, demonstrating the complexity and usability of the system.

In terms of hardware design, the ESP8266 provided intelligent control and data processing functions, while the SIM7020 module enabled NB-IoT connectivity for efficient and reliable remote communication. Combined with the DHT22 sensor, the system accurately gathered temperature and humidity information, providing comprehensive environmental sensing capabilities. This node design not only serves as the core of the system but also ensures the stable operation of the entire monitoring system.

For the software platform, we used tools such as Visual Studio Code, Arduino IDE, and IDEA, respectively, for writing and testing the front-end code, programming and flashing the hardware code, and developing the back-end code. The efficient use of these tools allowed us to successfully develop the various system modules, showcasing the team's expertise in IoT development. In addition, we successfully configured the cloud server, installed the database, and set up an MQTT server for message aggregation and transmission, which deepened our understanding of the integration of cloud computing and IoT and provided strong support for the system's scalability and data processing.

Moreover, we incorporated an LSTM model for predicting temperature and humidity. By training and predicting based on historical data, the LSTM model effectively forecasts future temperature and humidity variations, providing the system with forward-looking decision-making capabilities.

In summary, the successful implementation of this project not only applied the knowledge we have learned but also enhanced the overall capabilities of the team. From hardware design to front-end and back-end development, and the integration of temperature and humidity prediction, the team gained valuable IoT development experience and strengthened our ability to solve practical problems.

Keywords: NB-IoT ESP8266 module MQTT message queue Frontend and back- end separation Temperature and humidity monitoring LSTM model

一、 设计背景与目标

1.1 设计背景

环境温湿度监测在许多行业中具有重要意义，包括农业、物流、仓储、医疗等众多重要领域。通过实时监测环境的温度和湿度，可以提前预警并及时采取措施，保证设备的正常运行，提高生产效率，防止质量问题的发生。除此之外，全球气候变化和环境问题引起了广泛关注，气温升高、极端天气和自然灾害频发成为不容忽视的现实。

在当下，许多企业和机构依然采用传统的手工监测模式，投入数量有限的监测点， 并投入一定量的人力，导致监测数据出现获取不及时、准确性低等状况。这类方式存在一定的弊端，包括高昂的成本、效率低下、难以实现全面覆盖等问题。但随着对环境温湿度的要求不断提高，许多行业迫切需要一种可靠且智能化的环境温湿度监控报警和预测系统。

这种系统将基于物联网技术，通过部署一定数量的传感器设备，在关键位置实时监测环境的温度和湿度，通过数据采集和处理，不仅可以实时监测环境参数，还可以根据预设条件自动触发报警机制。此外，系统将利用机器学习和数据分析技术，对采集到的历史温湿度数据进行预测，提前预估未来一段时间的环境状态。通过这些预测，系统能够在温湿度可能超出安全范围之前进行预警，从而为用户提供实时的环境状态信息、报警提示和环境状态预测，帮助用户尽早发现异常情况并采取相应的措施。

1.2 设计目标

本次课程设计题目为《基于 NB-IoT 和 LSTM 的智能环境监测预测系统》，包含以下若干需要实现的设计目标。

1. 基于 NB-IoT 技术进行应用开发，实现一个完善的智能温湿度监控预测系统；
2. 掌握基本的物联网开发技术，包括无线节点的代码设计与写入、通过无线信道进行信息传输；
3. 熟悉 NB-IoT 的基本原理与常见硬件模块的使用方式；
4. 熟悉各类常见传感器的电气特性与使用方法，例如 DHT 系列传感器、蜂鸣器等；
5. 掌握基础全栈开发技术，学习常用前后端框架，以完成应用平台的开发；
6. 掌握基本机器学习模型，例如 ARIMA、LSTM，对测得的数据进行预测和分析；
7. 培养在实践中发现问题与解决问题的能力。

二、 课程设计器材

选择合适的硬件平台和软件平台可以为本次课程设计提供极大的设计便利，利于最终的智能温湿度监控预测系统的实现。

2.1 硬件平台

硬件平台的选用将综合考虑性能、稳定性、成本、使用门槛、平台拓展性等方面。节点的开发板方面，本次实验选用 ESP8266 模块，NB-IoT 联网方面选用 SIM7020 模块（测试阶段使用 Wifi），温湿度监测方面选用 DHT22 模块。这些器件的主要特点如下。

- ESP8266：ESP8266 是一款低成本、低功耗的开发模组，具有高性能的 Wi-Fi 模块，内部集成 MCU 以实现串口通信。选用该模块的原因包括：成本较低，试验代价较小；具有 Wi-Fi 模块，联网方便，便于开发过程中的信息传输调试。
- SIM7020：SIM7020 模块是一种低功耗、小型化的 NB-IoT 模块，专为物联网应用设计，可以用于远程设备的低功耗、长距离通信。具有低功耗、广覆盖、高连接性的特点。选用 SIM7020 模块可以实现设备与云端的可靠、高效的通信，使得温湿度监控预测系统可以在不同地点进行数据传输。
- DHT22：DHT22 是一种数字温湿度传感器，能够测量环境中的温度和湿度，并通过数字信号输出这些数据，是一种成本效益高且易于使用的温湿度传感器。它提供了对环境条件的准确监测，是制作低成本、高效率温湿度监控预测系统的理想选择

2.2 软件平台

- Arduino IDE：Arduino 是一种开源电子原型平台，包括硬件和软件，它使用简单的 C/C++ 语言编写，为物联网和嵌入式系统提供了一种易于使用的开发环境。使用其集成开发环境（IDE）能够非常快速地开发并测试物联网设备的硬件代码， 故本次课程设计主要使用 Arduino 作为硬件的开发平台。
- IDEA：IntelliJ: IDEA 是一款由 JetBrains 开发的 Java 集成开发环境，适用于后端的代码编写、测试、打包，有助于后端的开发效率。
- Visual Studio Code：VSCode 是一款轻量级的开源代码编辑器，支持多种编程语言，本次课程设计主要使用其进行前端代码的开发，它提供了丰富的前端插件， 简化了开发流程。
- Pycharm：PyCharm 是由 JetBrains 开发的一款强大的 Python 集成开发环境（IDE），专为 Python 编程设计。它提供了智能代码补全、调试工具、语法高亮、版本控制集成等功能，极大地提高了开发效率。本课程涉及主要使用他进行机器学习模型的训练和后续的数据分析预测。

- Mosquitto：Mosquitto 是一个开源的轻量级 MQTT 消息代理，专为物联网 (IoT) 环境设计。它支持 MQTT 协议，具备高效、低资源占用的特点，适合用于嵌入式设备和资源受限的系统。Mosquitto 配置简单，支持多种操作系统，提供安全的通信和访问控制功能。作为一个活跃的开源项目，它被广泛应用于设备间的实时消息传递，适用于智能家居、传感器网络等 IoT 场景。

三、 设计方案

3.1 思路分析

对于无线智能温湿度监控预测系统，需要考虑以下若干方面：不同位置的温湿度信息获取、温湿度信息的汇总与存储、数据的合理处理与呈现。

对于温湿度信息的获取，需要使用相关的温湿度信息监测模块，并连接到某一节点上，节点需要通过温湿度传感器获取温湿度信息，并对信息进行合理的处理，包括异常值的筛选、监测时间的记录。且每个节点之间是独立且可分辨的，可以互不干扰地进行温湿度信息的获取，通过多节点实现不同空间位置上的监测。对于温湿度信息的汇总，需要考虑节点之间的信息共享，因此需要网络的建立，本次课程设计需要使用窄带物联网进行通信，因此各节点需要配备 NB-IoT 的通信模块。除此之外，各传感节点中获取的信息各不相同，且具有异步特性，故考虑使用消息队列服务器进行信息的汇总处理，本次设计采用 Mosquitto 服务器作为消息汇总的中心，各节点共同订阅某一主题，并在获取到温湿度数据后将数据进行打包（添加节点编号、时间戳等信息），并统一通过 NB-IoT 发送至 Mosquitto 服务器的特定主题，以实现信息的共同发布与汇总。

对于数据的存储，考虑使用后端程序（本次课程设计选用 springboot 框架）实现， 与各传感节点类似，后端程序也应作为用户订阅 EMQX 服务器中的指定主题，但应作为信息的接收者，在接收到传感器的信息发布后进行消息的接收，并连接数据库（选用 MySQL）进行数据的存储。

对于数据的预测，考虑使用 python 实现，考虑到温湿度对一天内的时间敏感，因此使用 LSTM 模型对采集到的数据进行预测，我们将会先收集多天完整的数据，数据尽量详尽，且天气为阴天，防止太阳或者下雨对温湿度的过大影响。对于数据的处理与呈现，考虑使用前端 Web 应用进行报表的呈现，同时提供给用户操作数据的接口，包括阈值设定，动态报表呈现，数据的导出等，前端代码需向后端进行数据的请求，后端程序也应当根据前端的需求提供合理可用的接口。

3.2 系统架构

根据 3.1 节中所述的设计思路，可以进行系统整体的架构图的呈现，大致结构如图 3.1 所示。

为更好地说明系统整体的运行方式，考虑对以下过程进行流程分析：从某一节点采集数据开始直至数据呈现于用户。流程图如图 3.2 所示。

可以确定，硬件端与软件端之间的运行相对独立，即前文所提及的硬件端的信息发布具有异步性，因此软件段需要在运行后持续等待硬件层面的数据传输。

3.3 功能设计

在正式进行课程设计的开发前，我们进行了系统的系列需求分析，确定了一系列面向用户的功能。

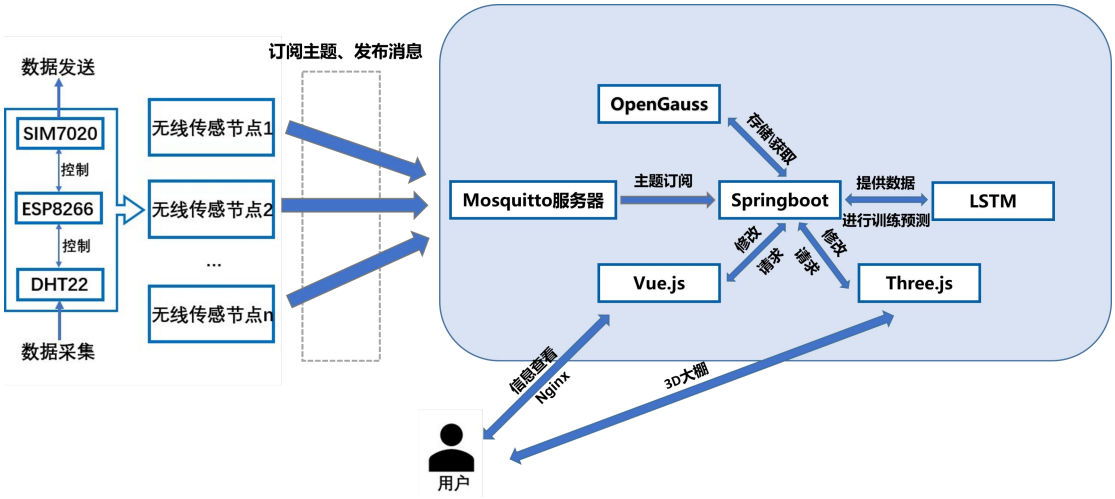


图 3.1 系统整体逻辑结构

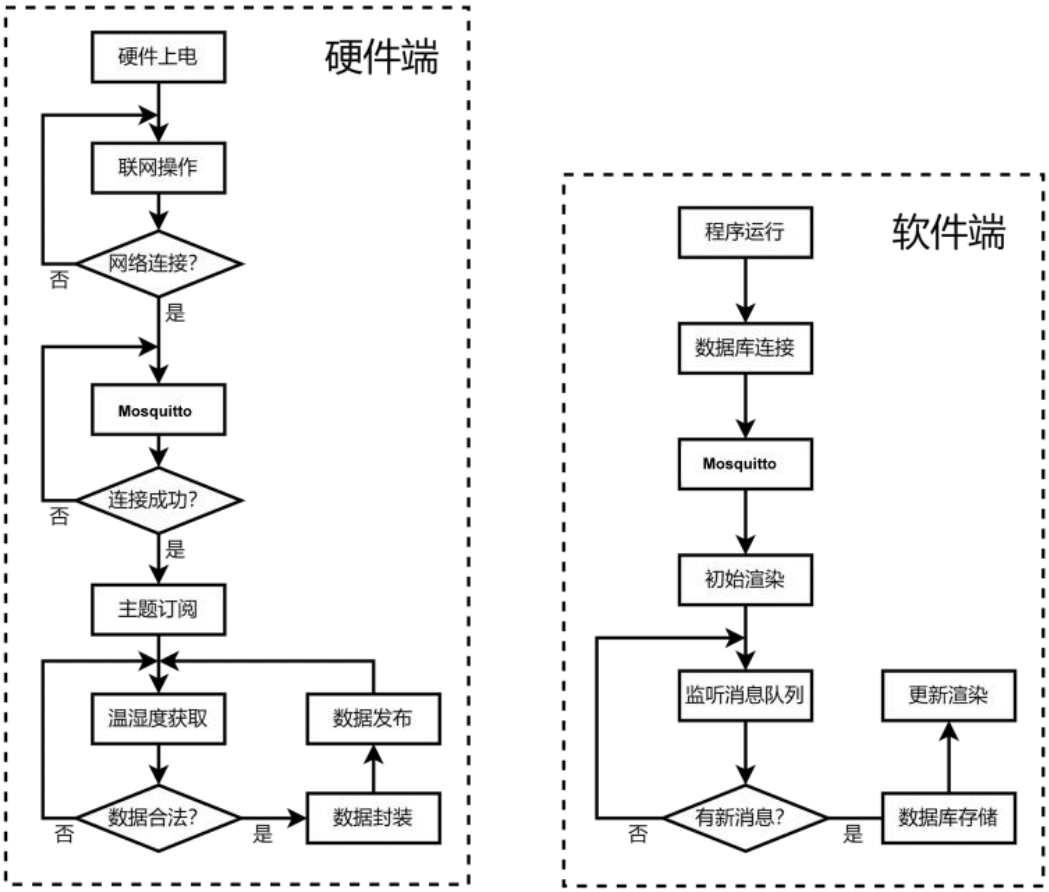


图 3.2 系统运行流程图

该系统的核心功能包括：实时监控与预测、数据分析与渲染呈现、异常报警设置与通知、历史数据查询、日志报告导出等部分。具体的功能描述如下表 3.1 所示。

表 3.1 系统功能需求表

功能概述	详细描述
实时监控预测	系统具备实时监测环境温湿度的功能，并且可以根据历史温湿度，精准预测接下来一段时间温湿度，帮助进行监控与预警
数据分析与渲染呈现	系统能够对采集到的温湿度数据进行分析，提供直观的趋势展示，例如二维折线、三维热力图等。
异常报警设置与通知	系统支持自定义的报警规则设置。温湿度数据超出设定的阈值将触发报警机制，并以短信、电子邮件等形式通知相关人员。

历史数据查询	系统将保存一定时间范围内的历史数据，可以通过用户界面查询历史温湿度数据
日志报告导出	系统根据指定的时间间隔进行报告的生成，并提供导出服务器，包括数据分析表、异常情况统计
设备管理	系统提供对所有监测设备的统一管理服务平台，供管理员进行设备管理、系统设置
3D大棚	系统提供一个仿真3D大棚，为用户根据温湿度只能进行设备管理，且3D模型可操纵，身临其境

3.4 技术栈选型

本次课程设计选用的技术栈列举如下。

- 服务器端：Spring boot、MyBatis、Spring Security、Mysql、Redis、Mosquitto
- Web 端: ES6、Vue、Vuex、Vue-router、Vue-cli、Axios、Element-ui、Three.js
- 硬件端：ESP、Arduino、SIM、DHT
- 预测模型：LSTM

四、 设计过程

设计实现包括以下若干步骤：服务器搭建、硬件端设计实现、前端页面设计、后端接口提供、LSTM 模型训练与预测。

4.1 服务器搭建

4.1.1 数据库搭建

考虑到无线传感器节点将获取大量温湿度数据，并对其进行分析、预测和可视化，因此我们考虑采用数据库存储对应数据，以便提供对应数据存储和接口。本次设计选用的数据库为 OpenGauss。为了方便部署和使用，我们使用 Docker 作为数据库启用管理工具。如图 4.1 所示，数据库服务启动。

```
sh-5.1# su opengauss


Welcome to 5.15.153.1-microsoft-standard-WSL2

System information as of time:  Sun Dec  1 22:49:13 CST 2024

System load:    0.33
Processes:      8
Memory used:    14.6%
Swap used:      0%
Usage On:       1%
Users online:   0
To run a command as administrator(user "root"),use "sudo <command>".
[opengauss@530bb0d84ad2 /]$ gsql
gsql ((openGauss 5.0.0 build ) compiled at 2023-04-18 09:47:39 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.
```

图 4.1 MySQL 安装与服务启动

由于使用的 Docker 容器具有对应的端口地址，因此可以使用私网 IP +端口的形式对该数据库进行访问连接，本次实验采用 DataStudio 作为数据设计的管理工具，使用 DataStudio 连接至云端数据库，首先通过 root 用户创建了一个名为 mysql_mqtt 的数据库，以存储后续的数据表，并作为后端连接的主数据库，进行数据表的设计，考虑到这类应用场景数据类型单一，只考虑设置两张数据表，分别为记录 record 与硬件信息表 hardware。

对于硬件信息表，其中包含字段为：硬件 ID（主键）、温度阈值上下限、湿度阈值上下限、硬件类型、硬件状态、收集数据总数、硬件描述，如图 4.2 所示。

#	名称	数据类型	长度/集合
1	**hardid**	**INT**	**10**
2	tempmax	FLOAT	
3	tempmin	FLOAT	
4	humimax	FLOAT	
5	humimin	FLOAT	
6	model	VARCHAR	50
7	state	BIT	1
8	datanum	INT	10
9	description	VARCHAR	50

图 4.2 硬件信息表 hardware 内容

对于记录表，其中包含字段为：记录 ID（主键）、来源设备（依赖于硬件

信息表）、时间戳信息、温度信息、湿度信息，如图 4.3 所示。

字段: 添加 删除 向上 向下

#	名称	数据类型	长度/集合
1	id	INT	10
2	froms	VARCHAR	50
3	timestamp	VARCHAR	50
4	temperature	VARCHAR	50
5	humidity	VARCHAR	50

图 4.3 记录表 record 内容

4.1.2 MQTT 服务器搭建

本次课设选用 MQTT 服务器作为消息队列服务器，考虑到维护和测试的方便，故自主进行 MQTT 服务器的搭建，与数据库服务器类似对外提供 Mosquitto 服务，接受各用户的订阅与消息共享。使用 1883 端口在本地上位机上完成 Mosquitto 服务的安装后，确认 Mosquitto 服务运行，如图 4.4 所示。

```
1733065320: mosquitto version 2.0.19 starting
1733065320: Config loaded from mosquitto.conf.
1733065320: Opening ipv6 listen socket on port 1883.
1733065320: Opening ipv4 listen socket on port 1883.
1733065320: mosquitto version 2.0.19 running
```

图 4.4 Mosquitto 服务运行

4.2 硬件端搭建

4.2.1 NB-IoT 联网

使用 SIM7020 模块进行，选用合适的 NB-IoT 的 SIM 卡，插入 SIM7020 模块中， 按照图 4.5 的形式将 SIM7020 模块与 ESP8266 开发板进行连接。

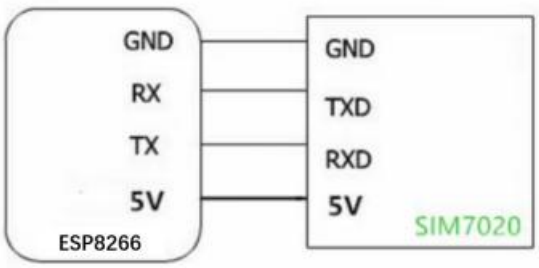


图 4.5 ESP8266 模块与 SIM7020 连接

通过 AT 指令集对 SIM7020 模块进行控制，通过 ESP8266 模块进行 AT 指令集的发送，相关代码内容如图 4.6 所示。

```
void connectnbiotmqtt() {
    Serial.begin(115200);
    Serial1.begin(115200);

    sim.reboot(true);
    sim.waitReady();

    // 服务器地址、端口号、超时时间
    String resp = sim.execCommand("AT+CMQNEW=\"10.83.11.4\",1883,12000,1024");
    Serial.println(resp);

    // NB-IOT设备唯一标识
    // 登录用户名和密码
    resp = sim.execCommand("AT+CMQCON=0,4 \"slb-sim7040\",\"admin\",\"kaiqi@666\"");
    Serial.println(resp);
}
```


图 4.6 通过 AT 指令集进行 NB-IoT 联网并连接至 MQTT 服务器

4.2.2 DHT22 模块安装

DHT22 模块仅包含 VCC、GND 和一个单线的数据 IO 口，将其连接到 ESP8266 上（引脚 4）。代码层面，加载 DHT 相关的库，并定义一个 DHT 对象，设置 DHT 类型与 DHT 的 IO 端口号。完成连接后可以通过 DHT 对象的 readHumidity() 和 readTemperature() 函数获取温湿度信息，若温湿度信息异常，则返回 NaN，需要对其异常值进行处理。代码内容如图 4.7 所示。通过 getTH 函数可以获取环境的温湿度，并存储于全局变量中，以供其余功能函数进行数据的获取。

```
#include <DHT.h>
#include <ESP8266WiFi.h>
#include <NTPClient.h>
#include <WiFiUdp.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

#define DHTPIN 4
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);
float currentTemperature = 0;
float currentHumidity = 0;
unsigned long currentUnixTime = 0;
//获取温湿度信息
void getTH() {
  delay(3000);
  float RH = dht.readHumidity();
  float T = dht.readTemperature();
  currentTemperature = T;
  currentHumidity = RH;
}

//封装成json格式
char messageBuff[1024];
void getJson() {
  StaticJsonDocument<1024> doc;
  JsonObject root = doc.to<JsonObject>();
  root["Temperature"] = currentTemperature;
  root["Humidity"] = currentHumidity;
  root["ID"] = 2;
  root["Timestamp"] = currentUnixTime;
  serializeJson(doc, messageBuff);
}
```

图 4.7 DHT22 的连接与温湿度获取转 Json

4.2.3 MQTT 消息发布

通过代码设置模块对某个主题进行订阅，这里选用的主题名称为“esp8266”，将 messageBuff 中的内容通过窄带物联网发送至 MQTT 服务器的指定主题中，其中 messageBuff 数组中的内容为 getJson 函数生成的，该函数用于将一系列硬件与温湿度信息封装成 Json 格式存储到 messageBuff 之中。代码内容如图 4.8 所示。

```
const char *topic = "esp8266";
//信息发布
void publishMQTTmsg() {
  getJson();
  if(mqttClient.publish(topic,messageBuff)) {
    Serial.println("Sended.");
  } else {
    Serial.println("Send Failed.");
  }
  memset(messageBuff,0,1024);
}
```

图 4.8 通过 NB-IoT 进行信息订阅与发布

4.2.4 WiFi 版本编写

在设计过程中，发现 NB-IoT 相关组件的连接不良，考虑编写一份使用 WiFi 联网的硬件代码，仅与 NB-IoT 版本在联网部分的部分代码有所差异。因此在测试过程中，也使用了基于 WiFi 联网的硬件获取方式。该部分内容主要进行了联网函数 connectWiFi() 函数的编写，其中设置了 WiFi 信息以供 ESP8266 进行循环连接。主函数内容如图 4.9 所示。由于篇幅受限，不对每一个函数的细节进行展示，以注释的形式说明其功能。

4.2.5 子功能测试

以上的内容实现了无线节点的温湿度信息获取，并能够通过窄带物联网进行 MQTT 服务器的连接，从而进行数据的发布，为测试该子功能的正确性，考虑使用 MQTTX 客户端进行同一主题的订阅，并观察获取的数据准确性。如图 4.10 所示，可以看到成功实现了节点环境温度的监测，并以 Json 格式将其发布到了 MQTT 服务器中，且其余订阅该主题的模块或软件均能够获取该信息。

```
//ESP8266WiFi连接信息 2.4G
const char* ssid = "kkaiqi";
const char* password = "15975300";
//WiFi连接
void connectWiFi() {
  WiFi.begin(ssid,password);
  while(WiFi.status()!=WL_CONNECTED) {
    delay(1000);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi Connected.");
  Serial.println("");
}
```

图 4.9 WiFi 版本的主函数



图 4.10 数据发布功能的测试

4.3 前端设计

4.3.1 页面布局

基于表 3.1 中所列的各项功能模块，使用 VSCode 与 Vue.js 前端框架进行前端界面的代码编写，并使用了 ElementUI 辅助进行页面的美化与设计。

网页界面的前端布局大体如图 4.11 所示。包含侧边导航栏，其中包含了首页、数据分析、密码修改、监控、历史数据、3D 大棚模型等子组件。

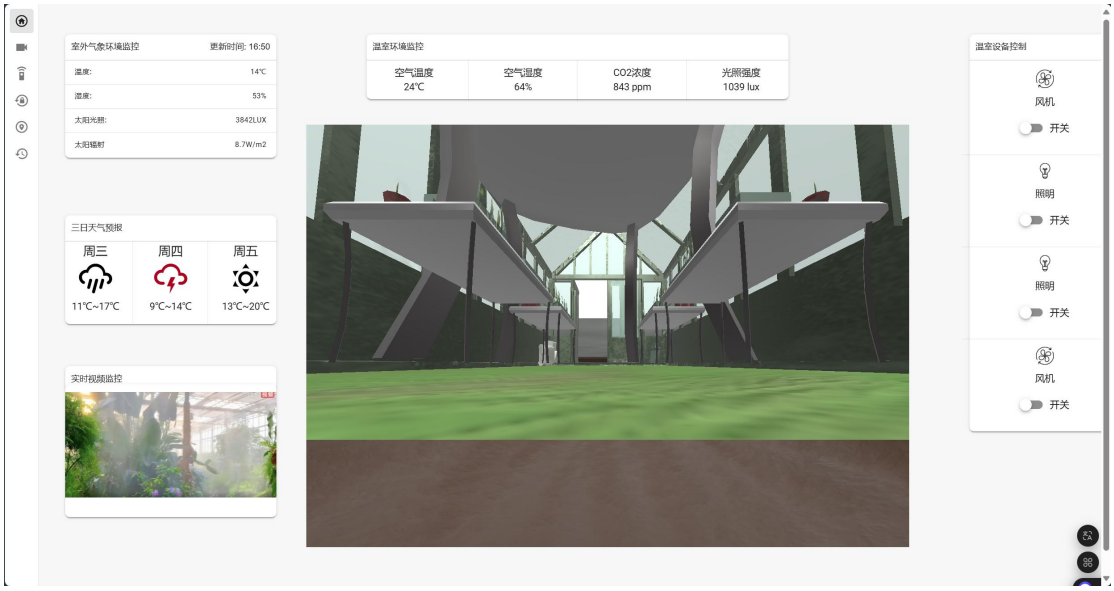


图 4.11 前端网页布局

对于页面的跳转，采用 vue-router 实现，路由信息如下图 4.12 所示，通过导航栏的点击，跳转到不同的路径，从而显示不同的组件内容。

4.3.2 数据请求

最开始数据尚未进行显示，需要前端向后端发送数据请求，并在前端使用 vue 的双向绑定功能进行数据渲染，数据请求层面，使用 axios 进行异步请求。具体地，首先进行 axios 的二次封装，设置请求的地址与端口（后端服务的地址与端口），再进行 api 接口的设置，需要定义请求方式（get、put、post、delete）与请求路径，如图 4.13 所示，为部分数据请求函数。

在各组件中，只需引入相应的数据请求函数，即可通过 axios 向指定的后端发送数据请求，并能够在 request 中获取数据的内容。后续需要在后端中依次提供这类数据的请求接口。

```
{
    path: '/management',
    name: 'Management',
    component: () => import('./views/management/index'),
    meta: {
        authRequired: true,
    },
},
{
    path: '/analysis',
    name: 'Analysis',
    component: () => import('./views/analysis/index'),
    meta: {
        authRequired: true,
    },
},
{
    path: '/histroydata',
    name: 'HistroyData',
    component: () => import('./views/histroy/index'),
    meta: {
        authRequired: true,
    },
},
{
    path: '/abouts',
    name: 'About',
    component: () => import('./views/abouts/index'),
    meta: {
        authRequired: true,
    },
},
```

图 4.12　使用 vue-router 实现页面跳转

```
export const getIotCount = () => {
    return http.get('api/getIotCount')
}
export const getAbnormalTempData = () => {
    return http.get('api/abnormaltempdata')
}
export const getAbnormalHumiData = () => {
    return http.get('api/abnormalhumidata')
}
export const editIot = (data) => {
    return http.put('/api/updateIot', data)
}
export const deleteIot = (params) => {
    return http.delete('/api/deleteIot/' + params)
}
export const appendIot = (data) => {
    return http.post('api/addIot', data)
}
```

图 4.13　前端部分数据请求

4.3.3 设备管理

在设备管理界面，进行了所有导入系统的无线节点信息的显示，包括唯一标识的设备 ID、设备型号、设备状态、发送数据流、温度阈值上下限、湿度阈值上下限、设备描述等，如图 4.14 所示。同时对每一个设备信息可以进行编辑、删除操作，在此界面中进行设备的添加操作，需注意设备 ID 需要非重复，系统将会自动监测是否存在该 ID 的设备发送的信息，若有则进行存储与整合，并实施渲染到前端以提示用户。

对于该部分的实现，主要采用 ElementUI 组件进行，数据主要来自于 hardware 表， 其中发送数据量的值筛选自 record 表。



图 4.14 设备管理界面设计



图 4.15 数据分析界面设计

4.3.4 数据分析

在数据分析界面，以动态报表的形式呈现所有正在工作的监测节点处的温湿度信息，并在图线中给出阈值范围，使得用户能够清晰地观察到温湿度所处的范

围，如图 4.15 所示。具体地，该部分采用 echarts 组件进行报表的呈现，前端每个一段时间（7s） 向后端发送数据请求，查询数据库中新增的数据条目，并添加到显示队列即可实现报表的动态更新，且请求到数据后需要进行数据的筛选，以实现多个设备之间的工作独立 性。

4.3.5 历史数据

在该界面中实现的内容较为简单，即将所有 records 表中的数据以表格的形式在前端进行呈现，并提供排序和筛选操作，使用户可以按照时间或者设备 ID 等信息查询指定时间段的数据信息，此外提供了报表导出与异常数据导出的功能，能够实现所有数据或异常数据的 excel 表格导出，方便后续的数据处理。该部分界面设计效果如图 4.16 所示。

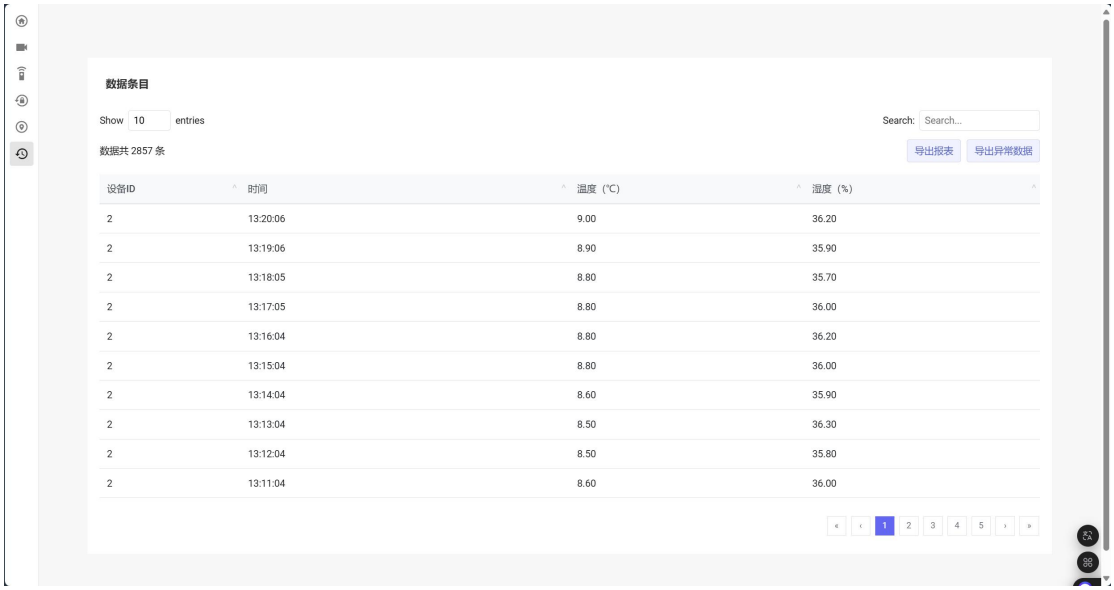


图 4.16 历史数据界面设计

4.3.6 3D 大棚组件

本组件使用 Three.js 构建 3D 大棚模拟仿真，动态模拟大棚内的物理环境、植物生长和光照变化等。首先，通过 Three.js 创建大棚的 3D 模型，包括大棚框架、屋顶、地面等基础结构，并应用纹理以提高视觉真实感。接着，模拟大棚内的物理环境，如温度、湿度和气流的变化，通过动态调整这些环境参数影响植物的生长。使用动画技术来模拟植物在不同环境下的生长过程，包括叶片和茎干的变化。光照方面，通过调整太阳光强度和角度，模拟一天 24 小时内的光照变化，影响植物的光合作用和生长状态。整个系统实现了大棚内环境与植物生长之间的实时交互与仿真。



图 4.17　3D 大棚组件界面

4.4　后端开发

前端的待渲染数据通过 axios 向后端请求，因此后端程序需要提供合理的数据请求接口，具体地，后端程序需要实现数据库的连接，以在获取温湿度信息后进行数据的转存，同时也需要进行 MQTT 服务器的连接与主题订阅，以消费者的形式进行消息的获取。

4.4.1　数据库的连接

由于后端框架采用 springboot 实现，需要在资源文件中进行相关配置的设置（ap- plication.yml），包括对数据库的信息配置，在 springboot 项目的初始化时已经选用添加了 MySQL Driver 依赖。由于数据库来自于 docker，根据 4.1.2 节中的数据库配置，在此需要设置 IP 为本地 IP，并设置数据库名为 mysql_mqtt，连接到 root 用户，并给定密码，如图 4.18 所示。

在完成配置后 springboot 程序将在运行时连接至该数据库，且后续 Mapper 层中的 SQL 语句将被发送到该数据库中执行，以实现数据的存储与查询。

```
spring:
  datasource:
    driver - class - name: com.mysql.cj.jdbc.Driver
    url: jdbc:mysql://10.83.11.4:3306/mysql_mqtt?userSSL=false
    username: root
    password: Mqtt@1912
```

图 4.18　后端程序连接数据库

4.4.2　MQTT 服务器的配置

若想要 springboot 能够连接 MQTT 服务器，首先需要导入 Maven 依赖，具体的依赖名为：spring-integration-mqtt，重新加载 Maven 依赖后即可实现相关资源的加载。

与数据库连接类似，MQTT 服务器的连接也需要在 application.yml 文件中进行相关配置的设置，包括服务器所在的 IP 与服务提供端口，对于 MQTT 服务端口，在 TCP 协议下均默认为 1883，此外还需要设定连接的 id 信息，该 id 需要保证在所有连接到目标服务器的客户端中非重复，最后需要设置订阅的主题。具体配置如下图 4.19 所示。

需注意的是，本次课程设计过程中搭建的 MQTT 服务器并未进行节点的认证操作， 因此这里的用户名和密码随机指定即可，服务器端并未对此进行认证。

```
mqtt:
  url: tcp://10.83.11.4:1883
  username: springboot
  password: 123456
  client:
    id: springboot_f07be17a_196825
    default:
      topic: esp8266
```

图 4.19　后端程序连接 MQTT 服务器的相关配置

4.4.3　服务器连接与数据获取

完成 MQTT 服务器的基本配置后，需要编写相关函数实现服务器的连接、主题订阅、消息监听与获取等操作，考虑设置一个 MQTT 消费者的控制类 MqttConsumer-Comfig，提供一些基本的连接、重连函数。

在加载 spring-integration-mqtt 依赖后，可以导入 MqttClient 类，该类的对

象实现了一个虚拟的 MQTT 客户，在实例化对象时，需要设置目标 MQTT 服务器的 IP 与端口信息，同时给定唯一标识的客户 id，这类信息可以在图 4.19 中的配置信息中获取， 可以通过 springboot 的 @Configuration 与 @Value 注解将配置信息加载到类中的私有变量中，以供类中函数的调用。

对于连接 MQTT 服务器的具体配置，可以通过 MqttConnectOptions 对象进行设置，包括用户名、密码、超时时长、保活时间、断开连接后的处理，最后通过 MqttClient 对象的 connect 函数进行配置的设置与连接操作执行即可完成连接。连接成功后再通过 subscribe 函数进行主题的订阅。

除此之外，最为重要的操作即为对 MqttClient 对象进行回调函数的设置，该依赖提供了完整的回调函数的设置格式，抽象于 MqttCallback 接口中，只需要实现其中的三个抽象函数即可，分别为 connectionLost（用于连接丢失后的处理操作设置）、messageArrived（获取信息后的处理）、deliveryComplete（完成发送后的后续处理）。对于 deliveryComplete 函数，本次课程设置无需进行实现；对于 connectionLost 函数，通过 MqttClient 对象的 isConnected 函数获取连接状态，并调用 reconnect 函数进行重连， 或者直接重新执行 connect 函数。

对于 messageArrived 函数，在主题中有新消息时，会以 MqttMessage 对象的形式将数据传入该回调函数，MqttMessage 对象提供了 getPayload 函数来获取其中加载的信息内容字符串，随后通过一系列 Java 操作将其转换为 Json 格式并获取所有信息。

在获取到所需的所有的数据后，考虑将数据进行封装并存入数据库，这里使用了 springboot 的三层架构，首先设置了实体类 Record，成员变量与数据表 record 中的字段一一对应，以用于数据的封装，后续定义 Service 层与 Mapper 层，在 Service 层中对外提供插入数据的服务，以 insertRecord 函数的形式呈现，具体在 Mapper 层中通过 SQL 语句进行数据的插入。SQL 语句即为：insert into record (froms ,times- tamp,temperature,humidty) values (froms ,timestamp,temperature,humidty)，其中 val- ues 中的数据来自于封装好的 Record 对象。

综上完成了 MQTT 服务器的连接、主题的订阅、消息的监听、数据的存储操作。核心代码内容如下：

```
public void
    connect()
    {try {
        //MQTT客户对象
        client = new MqttClient(hostUrl,clientId,new MemoryPersistence());
        //连接配置信息
        MqttConnectOptions options = new
        MqttConnectOptions();
        options.setCleanSession(false);
        options.setUserName(username);
        options.setPassword(password.toCharArray());
        options.setConnectionTimeout(100);
        options.setKeepAliveInterval(20);
        options.setWill("willTopic",(clientId +
        "与服务器断开连接").getBytes(),0,false);
        //重写回调函数内容 client.setCallback(new
        MqttCallback() {
        @Override //重连的处理方法
        public void connectionLost(Throwable throwable)
        { System.out.println("MQTT连接断开，发起重连
        ");
        try {
        if(client!=null && !client.isConnected())
        { client.reconnect();
        System.out.println("尝试重新连接");
        } else {
        client.connect(options); System.out.println("
        尝试建立新连接");
        }
        } catch (Exception e) { e.printStackTrace();
        }
```

```
}
@Override //消息获取后的处理方法
public void messageArrived(String topic,
MqttMessage message) throws Exception {
String msg = new String(message.getPayload());
ObjectMapper objectMapper = new ObjectMapper();
Map<String,Object> map =
objectMapper.readValue(msg, Map.class);
//获取消息中的关键信息
String Timestamp =
String.valueOf(map.get("Timestamp")); String
from = String.valueOf(map.get("ID"));
String Humidity =
String.valueOf(map.get("Humidity")); String
Temperature =
String.valueOf(map.get("Temperature"));
//信息封装
Record record = new Record();
record.setFroms(from);
record.setTimestamp(Timestamp);
record.setTemperature(Temperature);
record.setHumidity(Humidity);
//处理异常数据
if(Long.parseLong(Timestamp)<100) return;
//调用服务以插入数据
recordService.insertRecord(record);
}
@Override
public void
deliveryComplete(IMqttDeliveryToken
iMqttDeliveryToken) {}
});
//设置连接配置，随后发送连接
```

```
        client.connect(options);
        //订阅主题client.subscribe(defaultTopic,0);
        } catch (MqttException e) { e.printStackTrace();
        }
}
```

4.4.4 接口设计

该部分实现后端对前端提供的各类 api 接口的代码编写，采用 springboot 的三层架构，在 Controller 层提供对外的请求接口。以增添设备操作为例，需要前端发送 post 请求，后端层面通过 @PostMapping 注解设置请求路径为’/addIot’，并在函数参数处通过 @RequestBody 获取请求数据，即为待插入的 IoT 信息，随后调用 espService 中的 insertIot 服务，将待插入的对象传入，在其中继续调用 Mapper 层的特定函数，最后在 Mapper 层使用 SQL 语句进行数据插入即可。

需要注意的是，插入操作需要保证完整性依赖，即设备的 ID 不能重复，否则数据库层面会拒绝插入操作并以 Error 的形式返回后端，故后端代码需要使用 try-catch 语句捕获该异常，并向前端返回异常代码，若插入正常则返回成功代码。

该部分的代码内容如下。以此为例实现了前端所需的所有请求接口，完成了所有的前后端互通操作。

```
@PostMapping("/addIot")
public Result add(@RequestBody EspIoT
    iot) {try {
        espService.insertIot(iot);
    } catch (Exception e) {
        if(e instanceof
            DuplicateKeyException)
            {return Result.error("Insert
            Wrong");
        } else {
            return Result.error("System Wrong");
        }
    }
    return Result.success();
```

4.5 LSTM 模型

4.5.1 模型选择

本次课程设计主要采集的是温度与湿度，根据常识可知，温湿度具有强烈的周期性特性，适合时间敏感的模型，LSTM（Long Short-Term Memory）的优势在于其能够捕捉时间序列数据的长期依赖关系，同时有效处理数据中的周期性变化和短期波动。相较于其他复杂模型，也具有轻量级的特性，易于部署于分布式节点中。

4.5.2 模型训练

如图 4.20 所示，Pytorch 中已经封装了完整的 LSTM 模型,直接调用现有的网络模型，我们选用小时、分钟、温湿度四个特征进行训练。训练采用的数据集为校园中各个位置不同时段的平均温度，通过前文所述的 esp8266 和 DHT22 采集获得，如图所示。

```
class LSTMModel(nn.Module):  2 用法
    def __init__(self, input_size, hidden_layer_size, output_size, output_seq_len):
        super(LSTMModel, self).__init__()
        self.lstm = nn.LSTM( *args: input_size, hidden_layer_size, batch_first=True)
        self.fc = nn.Linear(hidden_layer_size, output_size * output_seq_len)
        self.output_size = output_size
        self.output_seq_len = output_seq_len

    def forward(self, x):
        lstm_out, _ = self.lstm(x)
        predictions = self.fc(lstm_out[:, -1, :])  # 只使用最后一个时间步的输出
        return predictions.view(-1, self.output_seq_len, self.output_size)
```

```
data['Hour_sin'] = np.sin(2 * np.pi * data['Timestamp'].dt.hour / 24)
data['Hour_cos'] = np.cos(2 * np.pi * data['Timestamp'].dt.hour / 24)
data['Minute_sin'] = np.sin(2 * np.pi * data['Timestamp'].dt.minute / 60)
data['Minute_cos'] = np.cos(2 * np.pi * data['Timestamp'].dt.minute / 60)
```

图 4.20 LSTM 架构和特征图

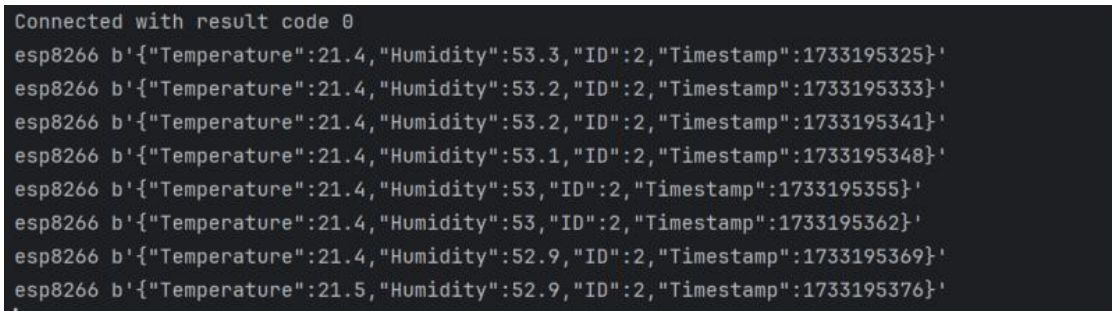


图 4.21　采集的数据流示意图

模型将经历 400 个 epoch，以保证在尽量拟合真实温湿度变化曲线的同时，不产生过拟合现象，并将训练完的模型保存到本地。当有新的数据读入时，系统将根据历史每 10 个数据对接下来 1 个时间点的数据的温湿度进行预测，每次向后预测 60 个时间点，预测结果如图 4.22 所示。

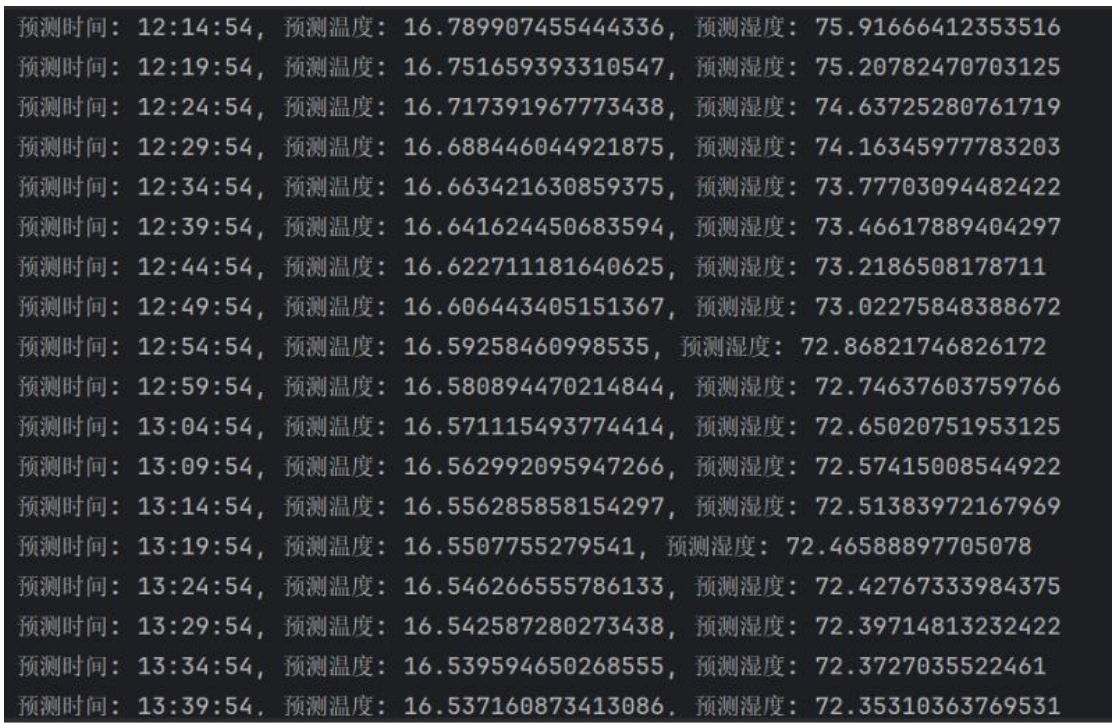


图 4.22　预测结果示意图

模型精准度如下所示

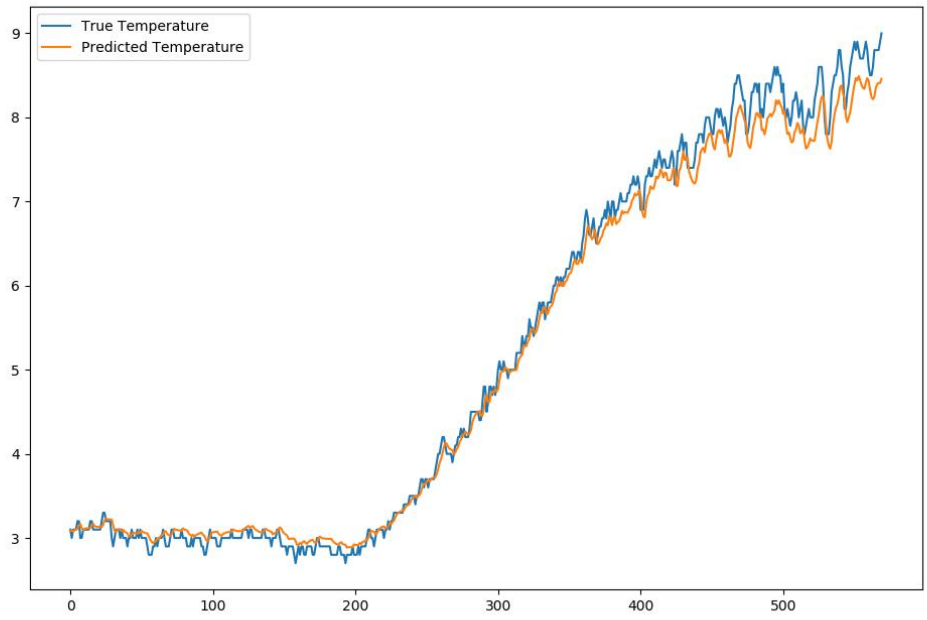


图 4.23　预测精准度示意图

五、　系统测试

5.1　系统部署

本系统运行于本地，但为云服务器预留了接口，可快速迁移至 ECS 上。后续的功能测试均在该部署完成的温湿度监控预测系统上进行。

5.2　功能测试

该部分将对各功能模块进行效果呈现。

5.2.1　主页视图

主页视图如图 5.1 所示。其中显示了连接设备数目、数据总条目数、平均温度、平均湿度、异常温度条数、异常湿度条数的概览信息，同时也提供了各设备实时温度的数值与简易视图，便于进行一些简单信息的查阅。

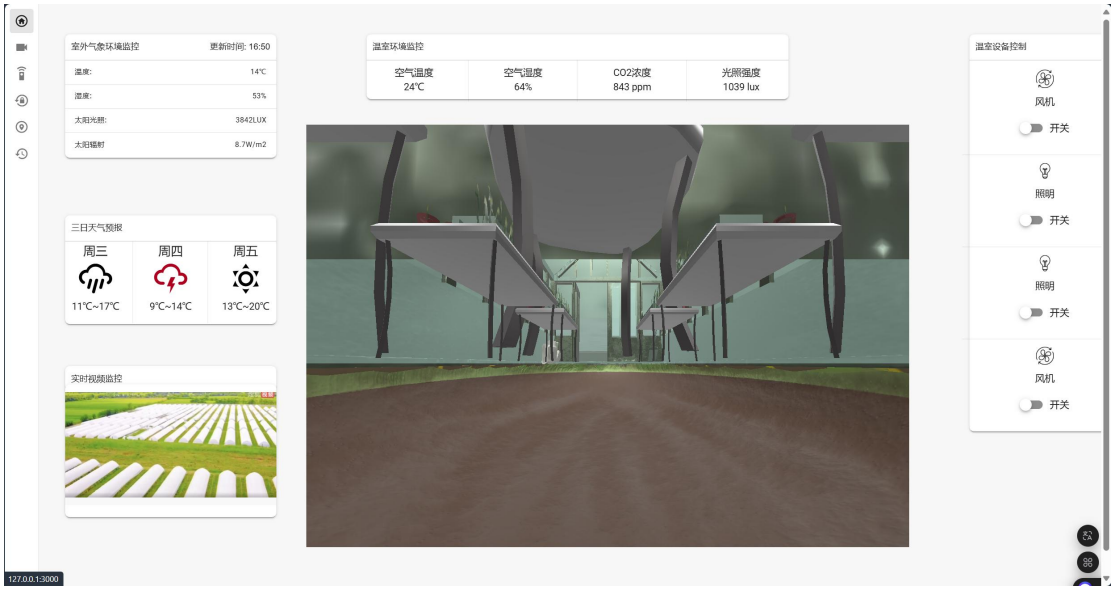


图 5.1 系统主页视图效果展示

5.2.2 动态视图

数据分析模块中进行了动态视图的显示，由于环境室温较为稳定，难以看出视图的动态性，因此考虑变化硬件节点的所处环境，可以看到数据发生了明显变化，但略延迟于实际时间点约 1s 左右，这部分延迟来自于传感节点的信息发布周期以及信息发布、获取所消耗的时间。视图效果如图 5.2 所示。

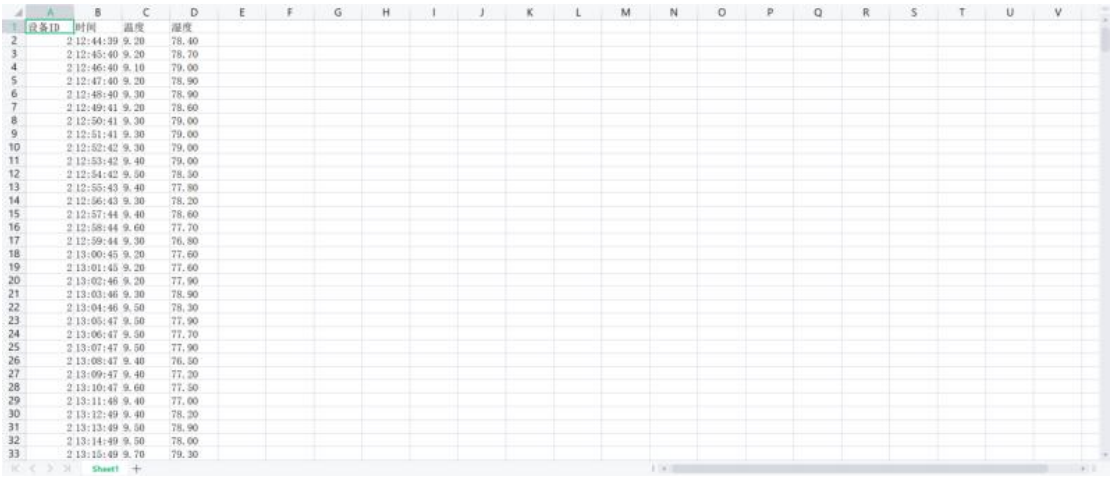
除此之外可以看到视图中展示了阈值范围与平均温度线，便于进行数据的监测与进一步的分析。



图 5.2 动态视图效果展示

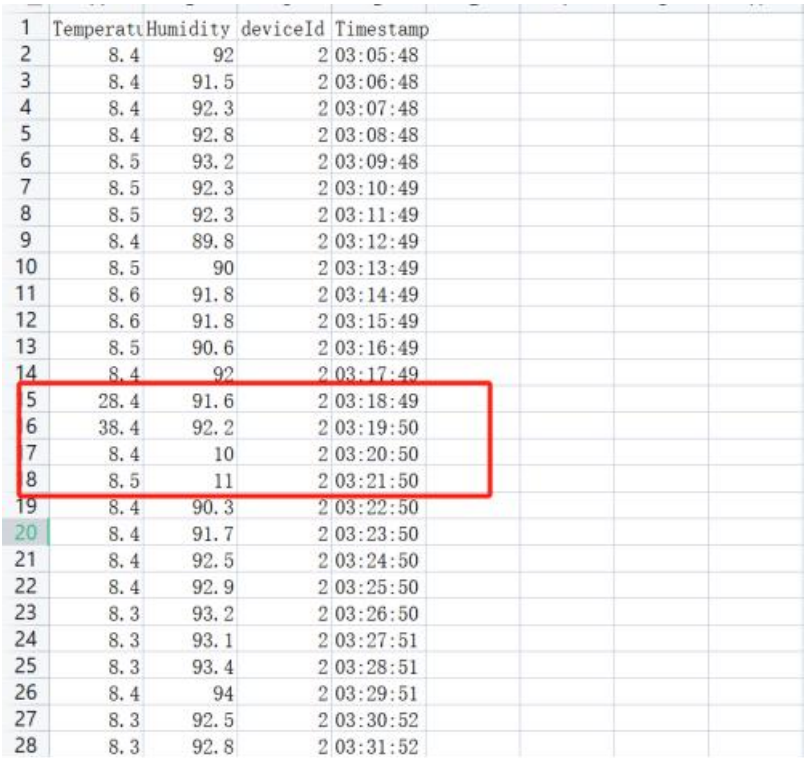
5.2.3 报表导出

对于历史数据模块，呈现了所有存储于数据库中的温湿度信息，并记录了其来源设备、具体时间等信息，同时也告知用户总共的数据条数，该数据与系统主页中呈现的一致。点击”导出报表“按钮，系统会自动下载一个 Excel 文件，其中包含了所有的温湿度信息，如图 5.3 所示；点击”导出异常数据“按钮，系统会自动下载一个额外的 Excel 文件，其中内容如图 5.4 所示，Excel 中包含了具体的时间、来源设备、温湿度信息、实际状态与正常的温湿度范围，以供使用者进行异常分析的参考。



设备ID	时间	温度	湿度
2	12:44:39	9.20	78.40
2	12:45:40	9.20	78.70
2	12:46:40	9.10	79.00
2	12:47:40	9.20	78.90
2	12:48:40	9.30	78.90
2	12:49:41	9.20	78.60
2	12:50:41	9.30	79.00
2	12:51:41	9.30	79.00
2	12:52:42	9.30	79.00
2	12:53:42	9.40	79.00
2	12:54:42	9.50	78.50
2	12:55:43	9.40	77.80
2	12:56:43	9.30	78.20
2	12:57:44	9.40	78.60
2	12:58:44	9.60	77.70
2	12:59:44	9.30	76.80
2	13:00:45	9.20	77.60
2	13:01:45	9.20	77.60
2	13:02:46	9.20	77.90
2	13:03:46	9.30	78.90
2	13:04:46	9.50	78.30
2	13:05:47	9.50	77.90
2	13:06:47	9.50	77.70
2	13:07:47	9.50	77.90
2	13:08:47	9.40	76.50
2	13:09:47	9.40	77.30
2	13:10:47	9.60	77.50
2	13:11:48	9.40	77.00
2	13:12:49	9.40	78.20
2	13:13:49	9.50	78.90
2	13:14:49	9.50	78.00
2	13:15:49	9.70	79.30

图 5.3 所有数据信息的报表导出



	Temperatu	Humidity	deviceId	Timestamp
1				
2	8.4	92	2	03:05:48
3	8.4	91.5	2	03:06:48
4	8.4	92.3	2	03:07:48
5	8.4	92.8	2	03:08:48
6	8.5	93.2	2	03:09:48
7	8.5	92.3	2	03:10:49
8	8.5	92.3	2	03:11:49
9	8.4	89.8	2	03:12:49
10	8.5	90	2	03:13:49
11	8.6	91.8	2	03:14:49
12	8.6	91.8	2	03:15:49
13	8.5	90.6	2	03:16:49
14	8.4	92	2	03:17:49
15	28.4	91.6	2	03:18:49
16	38.4	92.2	2	03:19:50
17	8.4	10	2	03:20:50
18	8.5	11	2	03:21:50
19	8.4	90.3	2	03:22:50
20	8.4	91.7	2	03:23:50
21	8.4	92.5	2	03:24:50
22	8.4	92.9	2	03:25:50
23	8.3	93.2	2	03:26:50
24	8.3	93.1	2	03:27:51
25	8.3	93.4	2	03:28:51
26	8.4	94	2	03:29:51
27	8.3	92.5	2	03:30:52
28	8.3	92.8	2	03:31:52

图 5.4 异常数据的导出（手动设置测试需要）

5.2.4 设备管理

设备管理模块中，由用户自行进行设备的添加，同时也可以进行基本的设备信息修改与删除操作，其中设备的修改界面如图 5.5 所示，其中设备 ID、设备型号、设备状态、数据条数等内容不允许进行修改，用户仅能够进行设备描述的内容修改以及温湿度阈值上下限的修改。

当完成对上下限修改后，该设备所对应的动态报表中的阈值返回以及异常判断标准也会随着发生改变。需注意设备 ID 会与实际硬件烧录中的 ID 信息一一对应。



图 5.5 设备的信息修改

5.2.5 3D 仿真大棚

3D 大棚组件中，左下角显示了当前环境的实时温湿度、光照、辐射信息，大棚中的设备会根据这些信息自动产生对应的控制信号，例如灯的开关，也可以手动控制设备的运行状况。



图 5.6 3D 仿真大棚

5.3 效果分析

通过一系列的功能测试，效果符合需求分析中的各项功能要求，由于 MQTT 服务器为自我搭建且与后端代码、数据库等部件位于同一台云服务器中，运行效率较高，且硬件通过 WIFI 进行联网，信息传输稳定且受地域干扰较小，并且是用 LSTM 对未来温湿度的预测也取得了较好的成果。本次课程设计的开发在功能实现层面体现了一定的复杂性与较高的可用性，符合预期。

六、 心得与体会

在这次的课程设计中，我们团队获得了丰富的物联网开发经验，取得了一系列可喜的成果。首先，通过对 ESP8266 的编程、ArduinoIDE 的灵活运用以及各类传感器模块的巧妙连接，我们深刻理解并掌握了物联网设备的开发技巧。这使得我们能够高效地实现温湿度监控预测系统的硬件层面，充分发挥了各种传感器的功能，为系统的数据采集提供了可靠的基础。

无线监测节点的设计为本次课设设计的重点，这一部分工作涉及到 ESP8266、SIM7020 模块以及 DHT22 模块的协同工作，为系统的 NBIoT 联网和 MQTT 服务器连接提供了可靠的基础。首先，通过 ESP8266 的灵活编程，我们成功实现了监测节点的智能控制和数据处理功能。ESP8266 作为微控制器，不仅令我们熟悉了硬件编程的精要，还为监测节点的智能化提供了可靠支持。其次，我们选用 SIM7020 模块，通过它实现了 NBIoT 的联网操作。这一步骤是整个监测系统的关键，使得监测节点能够远程连接至云端，实现了数据的远程传输。SIM7020 模块的低功耗、高效性能为我们的系统提供了可靠的网络通信能力。同时，我们结合 DHT22 模块，实现了对环境温湿度信息的高精度采集。DHT22 模块通过数字信号输出，为监测系统提供了可靠的环境数据，使得我们的智能监测节点能够更全面地感知环境的变化。最终，通过 MQTT 服务器的连接，我们成功将监测节点获取的数据整合并发送至云端。这不仅实现了监测功能，也为整个系统提供了远程信息采集的强大功能。通过这一设计，我们深刻体会到了无线监测节点在智能系统中的关键作用，以及其对于物联网应用的不可或缺性。

在软件方面，我们成功地熟悉和掌握了前后端的开发技巧。通过使用 Visual Studio Code 和 ArduinoIDE，我们能够迅速进行前端代码的编写和硬件代码的烧录，确保系统的复杂功能能够被可靠实现。与此同时，对于后端的开发，我们通过使用 IDEA，成功搭建了一个功能强大的系统后台，实现了数据的处理、存储和管理。这为系统的稳健性和可用性提供了有力的支持。

在项目的进一步优化中，我们引入了基于 LSTM 的温湿度预测功能。通过采

集到的历史温湿度数据，我们利用 LSTM 模型对未来一段时间内的温湿度变化进行了预测。LSTM 的引入使得系统不仅能够进行实时监测，还能提前预警可能的环境变化，增强了系统的智能化水平。这个功能通过 Python 和 TensorFlow/Keras 库实现，并与其他硬件和系统部分顺利集成。

服务器的搭建是我们团队在这次设计中的一项关键任务。我们成功地配置了云服务器，安装了数据库，并搭建了 MQTT 服务器进行消息的汇总和传递。这一系列操作不仅加深了我们对服务器架构的理解，也为系统的远程通信提供了坚实的基础。通过这一过程，我们对于云计算和物联网的结合有了更深刻的认识。

总的来说，这次课程设计不仅让我们掌握了丰富的技术知识，更锻炼了我们解决问题的能力。我们学会了发现并解决各种工程开发中可能遇到的问题，培养了团队协作和沟通的能力。最终，我们能够顺利完成课程设计的要求，这不仅是对我们所学知识的巩固，也是对我们工程开发能力的一次全面提升。这次设计不仅是知识的积累，更是团队协作和实践的宝贵经验。

参考文献

[1] 苗君臣, 王睿昊, 尤达等. 基于物联网的智能农业温室大棚监测系统 [J]. 物联网技术,2023,13(10):16-18.DOI:10.16667/j.issn.2095-1302.2023.10.004.

[2] 赵学作, 刘敏, 薛艳茹等. 基于树莓派的温湿度系统设计 [J]. 网络安全和信息化,2022(04):94-97.

[3] 杨海龙, 寇健, 温晓东等. 基于 ESP8266 的智能建筑温湿度检测系统设计 [J]. 河北建筑工程学院学报,2023,41(03):177-181+188.

[4] 胡浩鸣, 张胜利, 赵思等. 基于 ESP8266 的地窖环境监测系统设计 [J]. 现代信息科技,2023,7(22):150-155.DOI:10.19850/j.cnki.2096-4706.2023.22.033.

[5] 饶云. 基于 NBIOT 的工业环境监测平台设计与实现 [D]. 武汉工程大学,2021.DOI:10.27727/d.cnki.gwhxc.2019.000092.

[6] 徐毅. 基于 NBIOT 技术的环境监测系统的实验设计与实现 [J]. 科技创新与应用,2020(34):29-30.

[7] 王伟. 基于 NBIOT 的冷链运输温湿度监控管理系统设计与实现 [J]. 科技视界,2022(08):34-36.DOI:10.19694/j.cnki.issn2095-2457.2022.08.08.

[8] 王宇健.NBIOT 技术在工业环境监测中的应用 [J]. 现代工业经济和信息化,2022,12(10):96-97+99.DOI:10.16525/j.cnki.14-1362/n.2022.10.040.